Documentación Técnica

Y - Microblogging App

Concepto: Desarrollo de una red social multiplataforma
Nombre de la aplicación: Y
Redactado por: Pablo Vaz Castrillón
Curso: 2º CS Desarrollo de Aplicaciones Multiplataforma
Fecha de emisión: Marzo 2026

Índice de contenidos

1. Arquitectura del sistema
2. API
3. Base de datos
4. Aplicación Web
5. Aplicación Móvil
6. Aplicación de Escritorio
7. Seguridad
8. Despliegue
9. Guía de uso y mantenimiento
10. Mejoras futuras

1. Arquitectura del Sistema

La aplicación desarrollada es una red social basada únicamente en texto, similar a Twitter, donde los usuarios pueden publicar y visualizar publicaciones de otros usuarios.

El sistema sigue una arquitectura cliente-servidor, en la que las aplicaciones cliente consumen una API REST centralizada, la cual gestiona la lógica del sistema y el acceso a la base de datos.

Las aplicaciones cliente son:

- Aplicación web desarrollada con Laravel.
- Aplicación móvil desarrollada en Android Studio con Java.
- Aplicación de escritorio desarrollada con Windows Forms en C#.

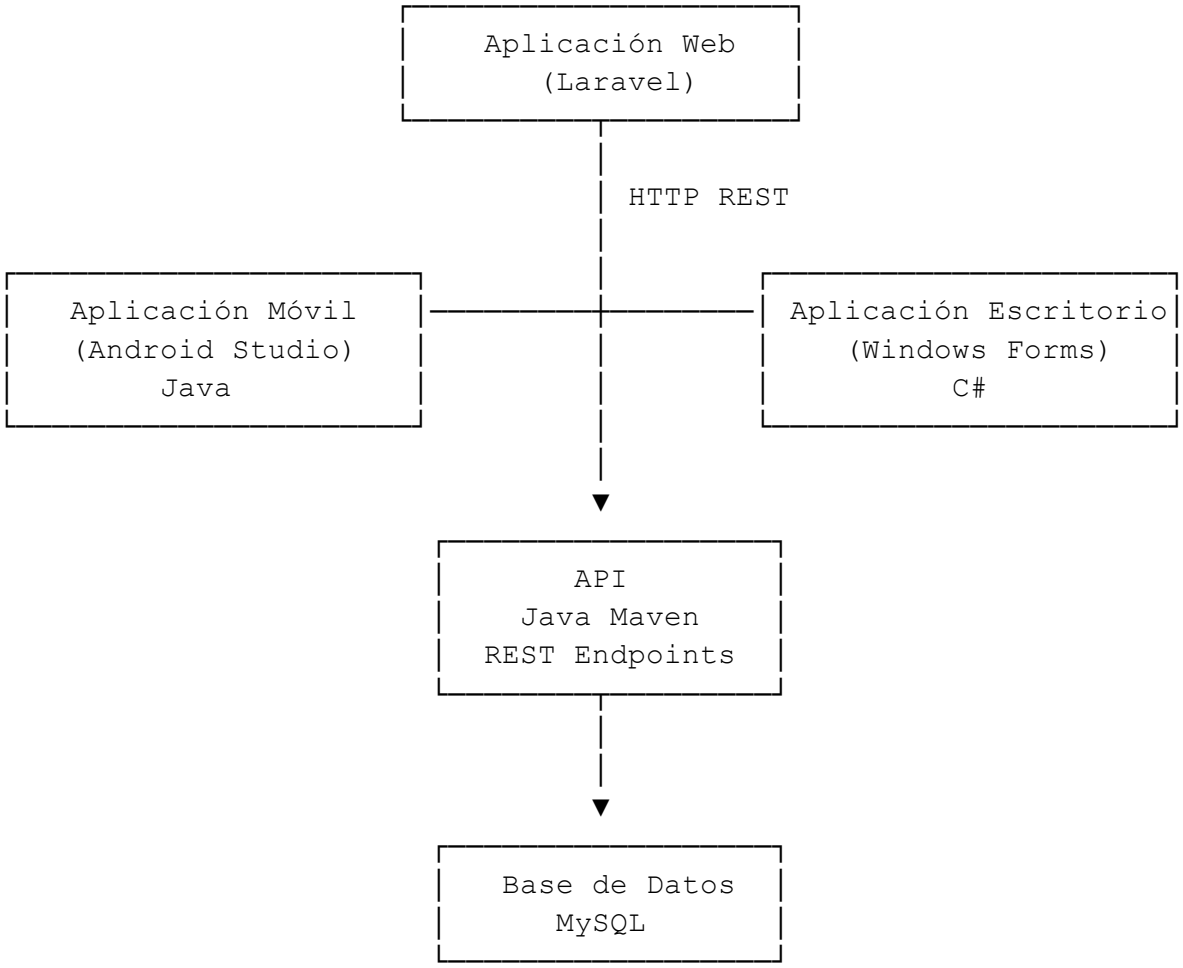
Todas las aplicaciones realizan peticiones HTTP a la API para realizar operaciones como autenticación, publicación de contenido o gestión de usuarios y seguidores.

1.1 Tecnologías utilizadas

Componente	**Tecnología**
API	Apache Tomcat, Java, Apache Maven
Aplicación Web	Laravel, HTML5, CSS3 y JavaScript
Aplicación Móvil	Android Studio, Java
Aplicación Escritorio	.NET Windows Forms, C#
Base de datos	MariaDB/MySQL (XAMPP en entorno local)

1.2 Diagrama de arquitectura del sistema

En el diagrama se puede observar cómo las tres aplicaciones cliente se comunican mediante peticiones HTTP con la API REST. Esta API actúa como intermediaria entre los clientes y la base de datos, centralizando la lógica de negocio y garantizando la coherencia de los datos.



2. API

La API REST constituye el núcleo del sistema, ya que centraliza la lógica del sistema y gestiona las peticiones realizadas por las distintas aplicaciones cliente.

Ha sido desarrollada en Java utilizando Apache Maven para la gestión del proyecto y desplegada en un servidor Apache Tomcat.

Las aplicaciones cliente se comunican con la API mediante peticiones HTTP, intercambiando datos en formato JSON.

2.1 Arquitectura de la API

La API sigue una arquitectura REST basada en recursos, donde cada entidad del sistema (usuarios, publicaciones, seguidores, etc.) representa un conjunto de endpoints.

Cada recurso está implementado mediante una clase Java que contiene los endpoints correspondientes. Por ejemplo, la clase `Users` implementa todas las operaciones relacionadas con la gestión de usuarios.

Cada método de la clase corresponde a un endpoint REST que gestiona una operación específica utilizando los distintos métodos HTTP `(GET, POST, PUT, DELETE)`.

2.2 Endpoints

Nota: Algunos endpoints incluyen acciones explícitas (como /create o /delete), aunque en una arquitectura REST pura estas operaciones se representarían únicamente mediante el uso de métodos HTTP sobre el recurso.

Users

A continuación se describen los endpoints disponibles para la gestión de usuarios:

Método	Endpoint	Descripción
POST	/users/register	Registrar un nuevo usuario
POST	/users/login	Inicio de sesión
GET	/users/{id}	Obtener los datos de un usuario
PUT	/users/update/{id}	Actualizar los datos de usuario
DELETE	/users/delete/{id}	Eliminar un usuario

Posts

A continuación se describen los endpoints disponibles para la gestión de publicaciones:

Método	Endpoint	Descripción
GET	/posts	Obtener todas las publicaciones
GET	/posts/{id_user}	Obtener las publicaciones de un usuario
GET	/posts/following/{id_user}	Obtener las publicaciones de usuarios seguidos por un usuario
POST	/posts/create	Crear una nueva publicación
DELETE	/posts/delete/{id}	Eliminar una publicación

Notifications

A continuación se describen los endpoints disponibles para la gestión de notificaciones:

Método	Endpoint	Descripción
GET	/notifications/{id_user}	Obtener las notificaciones de un usuario

Followers

A continuación se describen los endpoints disponibles para la gestión de seguidores:

Método	Endpoint	Descripción
POST	/followers/{id_follower}/follow/{id_following}	Seguir a un usuario
DELETE	/followers/{id_follower}/unfollow/{id_following}	Dejar de seguir a un usuario
GET	/followers/isFollowing/{id_follower}/{id_following}	Verificar seguimiento
GET	/followers/{id}/followersCount	Obtener número de seguidores
GET	/followers/{id}/followingCount	Obtener número de usuarios seguidos

2.3 Autenticación

La autenticación de usuarios se realiza mediante el envío de credenciales (email y contraseña) al endpoint de login.

Actualmente, el sistema valida las credenciales y devuelve la información del usuario autenticado.

Nota: No se utilizan tokens de sesión (como JWT), por lo que cada petición requiere autenticación previa desde el cliente.

2.4 Ejemplos de peticiones

Inicio de sesión

Permite autenticar usuarios utilizando su correo electrónico y contraseña.

Endpoint

```
POST /users/login
```

Petición

```
{
 "email": "usuario@email.com",
 "password": "123456"
}
```

Proceso interno

1. Se busca el usuario en la base de datos mediante el email proporcionado.
2. Si el usuario no existe, se devuelve error 404.
3. Si existe, se compara la contraseña enviada con el hash almacenado mediante BCrypt.
4. Si la verificación es correcta, se devuelve la información del usuario en formato JSON.

Respuesta correcta

```
{
 "id": 5,
 "username": "usuario1",
 "email": "usuario@email.com",
 "bio": "Desarrollador",
 "created_at": "2026-02-12"
}
```

Posibles errores

Código	Descripción
404	Usuario no encontrado
401	Contraseña incorrecta
500	Error interno del servidor

Registro de usuario

Permite crear una nueva cuenta en la aplicación.

Endpoint

```
POST /users/register
```

Petición

```
{
 "username": "usuario1",
 "email": "usuario@email.com",
 "password": "123456"
}
```

Proceso interno

1. Se comprueba si el nombre de usuario y el email están en uso.
2. La contraseña se cifra utilizando BCrypt.
3. Se inserta el nuevo usuario en la base de datos.

Respuesta correcta

```
201 Created
```

Posibles errores

Código	Descripción
409	Email ya registrado
409	Nombre de usuario en uso
500	Error interno del servidor

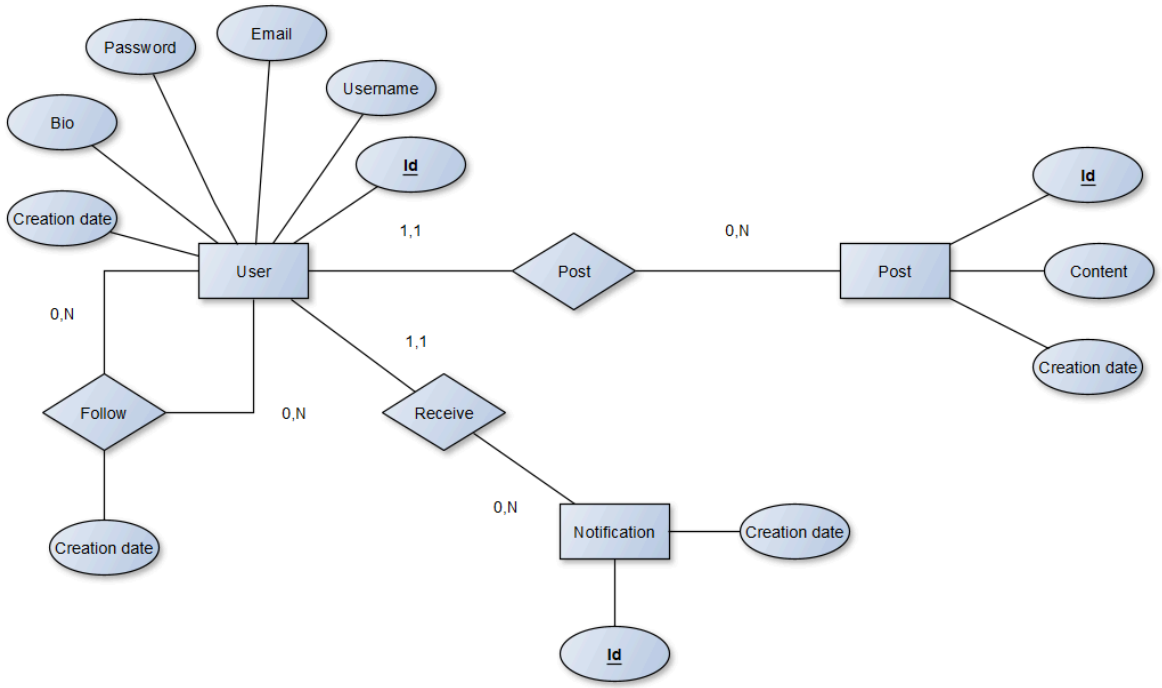
3. Base de Datos

La aplicación utiliza una base de datos relacional implementada en MariaDB (compatible con MySQL), desplegada por XAMPP, para almacenar la información de usuarios, publicaciones, relaciones de seguimiento y notificaciones.

El acceso a la base de datos se realiza desde la API desarrollada en Java mediante JDBC, utilizando consultas preparadas (Prepared Statements) para mejorar la seguridad y prevenir ataques de tipo SQL Injection.

La base de datos está estructurada en varias tablas relacionadas entre sí mediante claves primarias y foráneas, garantizando la integridad referencial.

3.1 Diagrama Entidad-Relación



La base de datos está compuesta por las siguientes tablas:

- **Users**: guarda la información de los usuarios.
- **Posts**: guarda la información de los posts.
- **Followers**: guarda información sobre las relaciones de seguimiento entre usuarios.
- **Notifications**: guarda la información de las notificaciones,actualmente solo existen notificaciones de tipo follow.

3.2 Tablas y Campos

Tabla Users

Campo	Tipo	Clave	Descripción
id	BIGINT UNSIGNED	Clave primaria	Identificador único del usuario
username	VARCHAR(255)	UNIQUE	Nombre de usuario
email	VARCHAR(255)	UNIQUE	Correo electrónico del usuario
password	VARCHAR(255)	—	Contraseña cifrada mediante BCrypt
bio	TEXT	—	Biografía del usuario
created_at	TIMESTAMP	—	Fecha de creación de la cuenta

Tabla Posts

Campo	Tipo	Clave	Descripción
id	BIGINT UNSIGNED	Clave primaria	Identificador de la publicación
id_user	BIGINT UNSIGNED	Clave foránea	Id del usuario que crea la publicación
content	TEXT	—	Contenido de la publicación
created_at	TIMESTAMP	—	Fecha de creación del post

Claves foráneas

Campo	Referencia
id_user	users.id

Comportamiento

Si un usuario se elimina, todas sus publicaciones también se eliminan (`ON DELETE CASCADE`).

Tabla Followers

Campo	Tipo	Clave	Descripción
id	BIGINT UNSIGNED	Clave primaria	Identificador de la relación
id_follower	BIGINT UNSIGNED	Clave foránea	Id del usuario que sigue
id_following	BIGINT UNSIGNED	Clave foránea	Id del usuario seguido
created_at	TIMESTAMP	—	Fecha de creación de la relación

Restricciones

Existe una clave única compuesta (id_follower, id_following)

Esto impide que un usuario pueda seguir a otro más de una vez.

Claves foráneas

Campo	Referencia
id_follower	users.id
id_following	users.id

Comportamiento

Si un usuario se elimina, también se eliminan automáticamente las relaciones de seguimiento.

Tabla Notifications

Campo	Tipo	Clave	Descripción
id	BIGINT UNSIGNED	Clave primaria	Identificador de la notificación
id_user	BIGINT UNSIGNED	Clave foránea	Usuario que recibe la notificación
id_follower	BIGINT UNSIGNED	Clave foránea	Usuario que ha generado la notificación
created_at	TIMESTAMP	—	Fecha de creación de la notificación

Claves foráneas

Campo	Referencia
id_user	users.id
id_follower	users.id

Comportamiento

Esta tabla almacena un registro de las relaciones de seguimiento, cuando se crea una relación de seguimiento en la tabla followers también se crea una notificación.

3.3 Reglas Importantes del Sistema

- Un usuario puede crear múltiples publicaciones.
- Un usuario puede seguir a muchos usuarios.
- Un usuario puede tener múltiples seguidores.
- Un usuario no puede seguirse a sí mismo.
- No se permite duplicar relaciones de seguimiento gracias a la clave única compuesta en la tabla `followers`.
- Si un usuario se elimina, también se eliminan automáticamente:

- Sus publicaciones
- Sus relaciones de seguimiento
- Sus notificaciones

3.4 Gestión de Migraciones

La gestión de la estructura de la base de datos se realiza mediante migraciones de Laravel, mientras que la API desarrollada en Java accede a los datos mediante JDBC sin modificar la estructura. Esto permite un control centralizado y reproducible de la base de datos, asegurando consistencia entre los distintos entornos (local, pruebas y producción).

Laravel también permite revertir migraciones mediante el comando php artisan migrate:rollback, lo que facilita deshacer cambios en la estructura de la base de datos.

Creación y actualización de la base de datos

1. Se generan archivos de migración en Laravel mediante el comando:

   `php artisan make:migration nombre_migracion`

2. Cada archivo define la creación o modificación de una tabla, incluyendo:

   - Columnas y tipos de datos
   - Claves primarias y foráneas
   - Restricciones de unicidad
   - Valores por defecto
   - Relación entre tablas (por ejemplo, `ON DELETE CASCADE` en claves foráneas)

3. Las migraciones se ejecutan con el comando: `php artisan migrate`

Esto aplica automáticamente todos los cambios definidos en los archivos de migración a la base de datos.

Beneficios de utilizar migraciones

- **Control de versiones de la base de datos:** cada migración queda registrada, facilitando revertir cambios o conocer la evolución del esquema.

- **Reproducibilidad:** cualquier entorno puede construir la base de datos completa ejecutando las migraciones, sin necesidad de SQL manual.

- **Mantenimiento sencillo:** para modificar la estructura de la base de datos (añadir nuevas tablas, columnas o índices) se crean nuevas migraciones, evitando la edición directa de la base de datos.

En caso de no querer usar Laravel

Sentencia de creación de la base de datos:

```
CREATE DATABASE `y_db` /*!40100 DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci */
```

Sentencias de creación de las tablas:

```
CREATE TABLE `users` (
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
  `username` varchar(255) NOT NULL,
  `email` varchar(255) NOT NULL,
  `password` varchar(255) NOT NULL,
  `bio` text DEFAULT NULL,
  `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
  PRIMARY KEY (`id`),
  UNIQUE KEY `users_username_unique` (`username`),
  UNIQUE KEY `users_email_unique` (`email`)
) ENGINE=InnoDB AUTO_INCREMENT=31 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci

CREATE TABLE `posts` (
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
  `id_user` bigint(20) unsigned NOT NULL,
  `content` text NOT NULL,
  `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
  PRIMARY KEY (`id`),
  KEY `posts_id_user_foreign` (`id_user`),
  CONSTRAINT `posts_id_user_foreign` FOREIGN KEY (`id_user`) REFERENCES `users` (`id`) ON DELETE CASCADE
) ENGINE=InnoDB AUTO_INCREMENT=40 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
```

```
CREATE TABLE `followers` (
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
  `id_follower` bigint(20) unsigned NOT NULL,
  `id_following` bigint(20) unsigned NOT NULL,
  `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
  PRIMARY KEY (`id`),
  UNIQUE KEY `followers_id_follower_id_following_unique`
(`id_follower`,`id_following`),
  KEY `followers_id_following_foreign` (`id_following`),
  CONSTRAINT `followers_id_follower_foreign` FOREIGN KEY (`id_follower`)
REFERENCES `users` (`id`) ON DELETE CASCADE,
  CONSTRAINT `followers_id_following_foreign` FOREIGN KEY (`id_following`)
REFERENCES `users` (`id`) ON DELETE CASCADE
) ENGINE=InnoDB AUTO_INCREMENT=42 DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_unicode_ci

CREATE TABLE `notifications` (
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
  `id_user` bigint(20) unsigned NOT NULL,
  `id_follower` bigint(20) unsigned NOT NULL,
  `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
  PRIMARY KEY (`id`),
  KEY `notifications_id_user_foreign` (`id_user`),
  KEY `notifications_id_follower_foreign` (`id_follower`),
  CONSTRAINT `notifications_id_follower_foreign` FOREIGN KEY (`id_follower`)
REFERENCES `users` (`id`) ON DELETE CASCADE,
  CONSTRAINT `notifications_id_user_foreign` FOREIGN KEY (`id_user`) REFERENCES
`users` (`id`) ON DELETE CASCADE
) ENGINE=InnoDB AUTO_INCREMENT=41 DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_unicode_ci
```

Integración con la API en Java

Aunque la API REST está desarrollada en Java con JDBC, la estructura de la base de datos se mantiene mediante las migraciones de Laravel. La API realiza únicamente operaciones de lectura y escritura de datos, sin alterar la estructura de las tablas.

4. Aplicación Web

4.1 Tecnología Utilizada

La aplicación web ha sido desarrollada utilizando el framework Laravel, un framework de desarrollo web basado en PHP que implementa el patrón de arquitectura MVC (Modelo-Vista-Controlador), permitiendo una clara separación de responsabilidades.

El desarrollo se ha realizado utilizando el editor Visual Studio Code, que permite trabajar de forma eficiente con proyectos PHP y frameworks modernos.

Las tecnologías principales utilizadas en la aplicación web son:

- Laravel como framework principal.
- PHP para la parte lógica
- HTML5 y CSS3 para la estructura y estilos.
- JavaScript para la interacción en el lado del cliente.

Laravel proporciona herramientas para la gestión de rutas, controladores, vistas y acceso a datos, facilitando el desarrollo de aplicaciones escalables, mantenibles y bien estructuradas.

4.2 Estructura del Proyecto

La aplicación fue planteada para seguir la arquitectura MVC, que separa la aplicación en tres componentes principales:

Componente	**Función**
Modelo	Gestión de datos e interacción con la base de datos
Vista	Interfaz que ve el usuario
Controlador	Gestiona la lógica entre modelo y vista

Debido a la simplicidad de la aplicación y al hecho de que los datos provienen directamente de una API externa, se optó por no utilizar modelos de Laravel.

En su lugar, los controladores realizan llamadas a la API y gestionan los datos en forma de arrays, que posteriormente se envían a las vistas. Esto simplifica la arquitectura y evita duplicar la lógica de acceso a datos ya implementada en la API.

La estructura del proyecto sigue la organización estándar de Laravel, adaptada a las necesidades de la aplicación:

```
.
├ app/
│   ├ Http/
│   │  └ Controllers/
│   │
│   └ Services/
│      └ ApiService.php
│
├ database/
│   └ migrations/
│
├ public/
│
├ resources/
│   └ views/
│      └ layouts/
│          └ app.blade.php
│
└ routes/
    └ web.php
```

En este esquema están presentes los archivos y directorios importantes para el funcionamiento del proyecto, a continuación se describen los directorios y archivos más relevantes:

Directorio app

Contiene la lógica principal de la aplicación.

Http/Controllers

Incluye los controladores encargados de gestionar las peticiones HTTP, procesar las solicitudes del usuario y devolver las respuestas correspondientes.

Services

Contiene clases de servicio que encapsulan lógica específica de la aplicación.

- **ApiService.php**
  Clase encargada de encapsular la comunicación con la API REST, centralizando las peticiones HTTP y evitando duplicidad de código en los controladores.

Directorio database

Contiene los archivos relacionados con la gestión y estructura de la base de datos.

migrations

Incluye las migraciones de la base de datos, donde se define la estructura de las tablas y sus modificaciones a lo largo del desarrollo.

Directorio public

Contiene los recursos públicos accesibles desde el navegador, como archivos estáticos, imágenes o scripts.

Directorio resources

Incluye los recursos de presentación de la aplicación.

views

Contiene las plantillas Blade utilizadas para generar las páginas HTML de la aplicación.

layouts

Define los layouts o plantillas base reutilizables.

- **app.blade.php**
  Layout principal compartido por las diferentes vistas dentro de la aplicación.

Directorio routes

Define las rutas de la aplicación, que establecen qué controlador se ejecuta para cada URL.

- **web.php**
  Archivo donde se definen las rutas web de la aplicación.

4.3 Conexión con la API

La aplicación web se comunica con la API REST mediante peticiones HTTP (`GET, POST, PUT y DELETE`), dependiendo de la operación a realizar. Estas peticiones permiten realizar operaciones como:

- Registro de usuarios
- Inicio de sesión
- Creación de publicaciones
- Gestión de seguidores

Las peticiones se envían desde la clase `ApiService.php` a los endpoints de la API desarrollada en Java y desplegada en Apache Tomcat.

La comunicación se realiza utilizando formato JSON, enviando los datos necesarios en el cuerpo de la petición y recibiendo las respuestas generadas por la API.

Las respuestas de la API son procesadas en los controladores y transformadas en estructuras de datos que posteriormente se envían a las vistas para su representación.

Este enfoque permite desacoplar completamente la capa de presentación de la lógica de negocio, facilitando el mantenimiento y la reutilización de la API por múltiples clientes.

5. Aplicación Móvil

5.1 Tecnología Utilizada

La aplicación móvil ha sido desarrollada utilizando Android Studio, el entorno de desarrollo oficial para aplicaciones Android.

El lenguaje de programación utilizado para la parte lógica es Java, ampliamente utilizado en el desarrollo nativo de aplicaciones Android.

Se trata de una aplicación nativa para Android, lo que permite un mejor rendimiento y una integración completa con el sistema operativo.

La aplicación permite a los usuarios interactuar con la red social desde dispositivos móviles, proporcionando funcionalidades como:

- Registro e inicio de sesión
- Visualización de un muro de publicaciones
- Creación y eliminación de publicaciones
- Seguimiento de otros usuarios
- Visualización de notificaciones de seguimiento

La aplicación sigue una arquitectura basada en la separación por capas (datos y presentación), facilitando la organización del código y su mantenimiento.

5.2 Estructura del proyecto

La estructura del proyecto sigue la organización estándar de aplicaciones Android:

```
app/src/main/
├ java/
│   └ com.example.y/
│          ├ adapters/
│          │
│          ├ data/
│          │   ├ models/
│          │   └ Api.java
│          └ ui/
│            ├ feed/
│            ├ login/
│            ├ notifications/
│            ├ otherprofile/
│            ├ post/
│            ├ profile/
│            └ MainActivity.java
└ res/
    ├ drawable/
    │
    ├ layout/
    │
    ├ menu/
    │  └ bottom_menu.xml
    │
    ├ navigation/
    │  └ main_graph.xml
    │
    └ values/
       ├ colors.xml
       ├ strings.xml
       └ themes.xml
```

A continuación una descripción de los directorios, con lo que contienen y su funcionalidad brevemente explicada.

Capa de datos

data

Gestiona el acceso a datos de la aplicación.

- **models**
  Contiene las clases que representan las entidades del sistema (usuarios, publicaciones, notificaciones), así como una clase singleton (Session) que gestiona el estado del usuario autenticado.

- **Api.java**
  Clase encargada de gestionar la comunicación con la API REST, centralizando las peticiones HTTP y el tratamiento de las respuestas en formato JSON.

Capa de interfaz

La aplicación utiliza una combinación de Activities y Fragments, donde la Activity principal actúa como contenedor de los distintos fragments que representan las pantallas.

La navegación entre fragments se gestiona mediante el componente Navigation, definido en el archivo main_graph.xml, lo que permite estructurar de forma clara el flujo de la aplicación.

adapters

Incluye los adaptadores utilizados por RecyclerView y ViewPager, encargados de enlazar los datos con las vistas.

ui

Contiene los componentes de interfaz de usuario (Activities y Fragments) organizados en carpetas independientes para cada parte de la aplicación.

- **feed**
  Fragment encargado de mostrar las publicaciones disponibles en el feed.

- **login**
  Activities de inicio de sesión y registro de usuarios.

- **notifications**
  Fragment que muestra las notificaciones del usuario.

- **otherprofile**
  Fragment utilizado para visualizar el perfil de otros usuarios.

- **post**
  Activity destinada a la creación de nuevas publicaciones.

- **profile**
  Fragment que muestra el perfil del usuario de la sesión.

- **MainActivity.java**
  Activity principal que gestiona la navegación entre fragments tras el inicio de sesión.

Resources

Contiene los recursos de la aplicación.

drawable

Recursos gráficos como iconos, imágenes y shapes.

layout

Archivos XML que definen la estructura de las interfaces de usuario.

menu

Define los menús de la aplicación

- bottom_menu.xml: menú utilizado en la barra de navegación inferior.
- menu_logout.xml: menú de un solo item utilizado para cerrar sesión.

navigation

Archivos de configuración de navegación entre fragments.

- main_graph.xml: grafo de navegación principal de la aplicación.

values

Archivos de valores reutilizables.

- colors.xml: definición de colores usados en la aplicación.
- strings.xml: cadenas de texto utilizadas en la interfaz.
- themes.xml: configuración del tema visual de la aplicación.

5.3 Conexión con la API

La aplicación móvil se comunica con la API mediante peticiones HTTP (GET, POST, DELETE), enviando y recibiendo información en formato JSON.

Las peticiones a la API se realizan de forma asíncrona para evitar bloquear el hilo principal de la aplicación, garantizando una experiencia de usuario fluida.

Las respuestas de la API son procesadas y transformadas en objetos del modelo, que posteriormente se utilizan para actualizar la interfaz de usuario.

Las clases encargadas de la comunicación con la API gestionan las siguientes operaciones, entre otras:

- Autenticación de usuarios
- Obtención de publicaciones
- Creación de publicaciones
- Gestión de seguidores
- Obtención de notificaciones

6. Aplicación de Escritorio

6.1 Tecnología Utilizada

La aplicación de escritorio ha sido desarrollada utilizando Visual Studio 2022 como entorno de desarrollo.

El lenguaje de programación utilizado es C#, junto con el framework .NET y la tecnología Windows Forms para la creación de interfaces gráficas de usuario.

Se trata de una aplicación de escritorio nativa para sistemas Windows, que permite a los usuarios interactuar con la red social sin necesidad de un navegador web.

La aplicación proporciona funcionalidades como:

- Registro e inicio de sesión
- Visualización de un muro de publicaciones
- Creación y eliminación de publicaciones
- Seguimiento de otros usuarios
- Visualización de notificaciones de seguimiento

6.2 Estructura del proyecto

La aplicación de escritorio sigue una arquitectura modular basada en la separación de responsabilidades, lo que permite organizar el código de forma clara, mantenible y escalable.

La estructura del proyecto se organiza en los siguientes directorios:

```
.
├ Controllers/
│
├ Forms/
│   └ Controls/
│
├ Models/
│
├ Resources/
│
├ Services/
│  ├ ApiService.cs
│  └ Session.cs
│
└ Program.cs
```

A continuación se describen los componentes principales del proyecto:

Controllers

Incluye las clases encargadas de la gestión de peticiones y respuestas hacia la API REST.

Su función principal es actuar como capa de abstracción entre la aplicación y la API, manejando la serialización de datos a JSON y la deserialización de las respuestas obtenidas.

Forms

Contiene los formularios (Windows Forms) que representan la interfaz gráfica de la aplicación.

Cada formulario gestiona la interacción con el usuario y muestra la información recibida desde los controladores.

Controls

Incluye controles personalizados reutilizables utilizados en distintos formularios, lo que permite mantener una interfaz consistente y evitar duplicidad de código.

Models

Contiene las clases que representan las entidades del sistema (usuarios, publicaciones, notificaciones, etc.).

Estos modelos se utilizan tanto para mapear las respuestas de la API como para estructurar los datos dentro de la aplicación.

Resources

Almacena los recursos gráficos de la aplicación, como iconos, imágenes y otros elementos visuales utilizados en la interfaz.

Services

Contiene las clases encargadas de encapsular la lógica de acceso a datos y la gestión del estado de la aplicación.

ApiService.cs

Clase responsable de centralizar la comunicación con la API REST, gestionando las peticiones HTTP y el procesamiento de las respuestas en formato JSON.

Este enfoque evita duplicidad de código y facilita el mantenimiento de la aplicación.

Session.cs

Clase encargada de gestionar la sesión del usuario autenticado, almacenando información relevante como el identificador o los datos del usuario durante la ejecución de la aplicación.

Program.cs

Archivo principal de la aplicación, responsable de iniciar la ejecución y lanzar el formulario inicial.

6.3 Conexión con la API

La aplicación de escritorio se comunica con la API mediante peticiones HTTP realizadas en los controladores y llamadas desde la clase `ApiService.cs`, enviando y recibiendo información en formato JSON.

Las clases encargadas de la comunicación con la API gestionan las siguientes operaciones, entre otras:

- Autenticación de usuarios
- Obtención de publicaciones
- Creación de publicaciones
- Gestión de seguidores
- Obtención de notificaciones

7. Seguridad

La seguridad es un aspecto fundamental en el desarrollo de aplicaciones que gestionan información de usuarios. En este proyecto se han implementado diferentes mecanismos de seguridad para proteger los datos, garantizar la integridad de la información y evitar accesos no autorizados.

Entre las principales medidas aplicadas se encuentran el cifrado de contraseñas, la prevención de ataques de inyección SQL, la validación de datos y el control de integridad en la base de datos.

7.1 Cifrado de contraseñas

Las contraseñas de los usuarios no se almacenan en texto plano en la base de datos. En su lugar, se utiliza la biblioteca jBCrypt, que implementa el algoritmo bcrypt para generar un hash seguro de la contraseña.

El algoritmo bcrypt incluye un mecanismo de salting automático, lo que incrementa la seguridad frente a ataques de diccionario y fuerza bruta.

Durante el registro de usuario, la contraseña introducida se cifra antes de guardarse en la base de datos:

```
String hashedPassword = BCrypt.hashpw(req.getPassword(), BCrypt.gensalt());
```

En el proceso de inicio de sesión, la contraseña proporcionada por el usuario se compara con el hash almacenado mediante el método:

```
BCrypt.checkpw(password, hashedPassword);
```

Este mecanismo garantiza que, incluso si la base de datos fuera comprometida, las contraseñas reales de los usuarios no puedan ser recuperadas fácilmente.

7.2 Prevención de SQL Injection

Para evitar ataques de SQL Injection, la API utiliza consultas preparadas (Prepared Statements) al interactuar con la base de datos.

Las consultas preparadas permiten separar la estructura de la consulta SQL de los datos introducidos por el usuario.

Este mecanismo evita que los datos introducidos por el usuario sean interpretados como parte de la consulta SQL.

Ejemplo de consulta preparada:

```
PreparedStatement ps = conexion.prepareStatement(
   "SELECT * FROM users WHERE email=?"
);
ps.setString(1, req.getEmail());
```

Este enfoque impide que un usuario malintencionado pueda insertar código SQL dentro de los campos de entrada.

7.3 Integridad y consistencia de datos en la base de datos

La base de datos implementa diversas restricciones que ayudan a mantener la integridad de la información.

Entre ellas destacan:

Claves únicas

En la tabla `users` se utilizan restricciones `UNIQUE` para evitar duplicados en campos importantes como:

- Nombre de usuario
- Correo electrónico

Esto garantiza que cada usuario tenga un identificador único dentro del sistema.

Claves foráneas

Las relaciones entre tablas se controlan mediante claves foráneas, que aseguran que los registros relacionados sean válidos.

Por ejemplo:

- Las publicaciones deben pertenecer a un usuario existente
- Las relaciones de seguimiento solo pueden darse entre usuarios válidos

Eliminación y actualización en cascada

En varias relaciones se utiliza la opción:

```
ON DELETE CASCADE, ON UPDATE CASCADE
```

Esto permite que, al eliminar o actualizar un usuario, se eliminen o actualicen automáticamente todos los registros relacionados, como:

- Publicaciones
- Relaciones de seguimiento
- Notificaciones

De esta manera se evita que queden registros huérfanos o desactualizados en la base de datos.

7.4 Validación de datos

La API valida los datos antes de almacenarlos en la base de datos. Por ejemplo:

- Se comprueba si un correo electrónico ya está registrado
- Se verifica que el nombre de usuario no esté en uso

Estas comprobaciones evitan inconsistencias y posibles errores en el sistema.

Además de estas comprobaciones, se valida el formato de los datos introducidos (por ejemplo, formato de email o longitud de los campos) para evitar datos inválidos o maliciosos.

7.5 Gestión de sesiones

La gestión de la sesión de usuario se realiza en las aplicaciones cliente, almacenando la información del usuario autenticado tras un inicio de sesión correcto.

En la aplicación móvil y de escritorio se utiliza una clase Session para mantener el estado del usuario durante la ejecución.

Actualmente, el sistema no utiliza tokens de autenticación (como JWT), por lo que la autenticación se basa en el control de sesión en el cliente.

7.6 Buenas prácticas aplicadas

Durante el desarrollo del proyecto se han aplicado diversas buenas prácticas de seguridad:

- Almacenamiento seguro de contraseñas mediante bcrypt.
- Uso de consultas preparadas para prevenir SQL Injection.
- Validación de datos de entrada.
- Uso de restricciones en la base de datos para garantizar la integridad de los datos.
- Separación de la lógica de negocio en una API REST centralizada.
- Control de sesión en las aplicaciones cliente.

Estas medidas contribuyen a mejorar la seguridad global del sistema y a proteger la información de los usuarios frente a accesos no autorizados.

8. Despliegue

El despliegue del proyecto abarca la API REST y las tres aplicaciones cliente (web, móvil y escritorio), que funcionan como consumidores de dicha API.

A continuación se describen los requisitos, pasos de instalación y ejecución de cada componente, así como las consideraciones necesarias para su correcto funcionamiento en distintos entornos.

Para el correcto funcionamiento de la aplicación es necesaria la base de datos, puede crearse con las migraciones del proyecto de laravel de la app web, o, ejecutando las sentencias de creación que figuran en el apartado 3.4.

8.1 Despliegue de la API

La API REST está desarrollada en Java con Maven y se despliega en un servidor Apache Tomcat. Puede ejecutarse tanto en entorno local como en un servidor remoto.

Requisitos previos

- Java JDK 17 o superior instalado
- Maven instalado y configurado
- MariaDB/MySQL ejecutándose en el puerto definido (`3306` por defecto)
- Base de datos creada
- Servidor Tomcat (versión 9 o superior)

Pasos de despliegue

1. Clonar el proyecto API desde el repositorio.
2. Construir el proyecto utilizando Maven:

   ```
   mvn clean package
   ```

3. Copiar el archivo `.war` generado en la carpeta `target/` a la carpeta `webapps/` de Tomcat. El archivo .war generado contiene la aplicación empaquetada lista para ser desplegada en el servidor Tomcat.

4. Iniciar Tomcat:

```
catalina.bat start      # Windows

./catalina.sh start     # Linux / macOS
```

5. Acceder a la API mediante la URL :

   `http://localhost:8080/Yapi/rest`
   (donde Yapi corresponde al nombre del proyecto desplegado en Tomcat)

6. Verificar el correcto funcionamiento con el endpoint de prueba:

   `GET /users/ping`

Si devuelve pong, la API está operativa.

8.2 Despliegue de la aplicación web

La aplicación web está desarrollada con Laravel y requiere un servidor local o remoto que soporte PHP.

Laravel permite configurar distintos entornos (desarrollo, producción) mediante el archivo .env, facilitando la adaptación de la aplicación a diferentes contextos de ejecución.

Requisitos previos

- PHP 8 o superior
- Composer
- Servidor web (Apache/Nginx)
- Base de datos MariaDB/MySQL con las migraciones aplicadas

Pasos de despliegue

1. Clonar el proyecto web desde el repositorio.

2. Instalar dependencias con Composer:

   ```
   composer install
   ```

3. Configurar el archivo `.env` con los datos de la base de datos y la URL de la API Java:

   ```
   DB_CONNECTION=mysql
   DB_HOST=127.0.0.1
   DB_PORT=3306
   DB_DATABASE=y_db
   DB_USERNAME=root
   DB_PASSWORD=
   API_URL=http://localhost:8080/Yapi/rest
   ```

4. Ejecutar migraciones para crear las tablas:

   ```
   php artisan migrate
   ```

5. Iniciar el servidor:

   ```
   php artisan serve
   ```

6. Acceder desde el navegador a:

   ```
   http://localhost:8000
   ```

La aplicación web funcionará como cliente de la API REST.

8.3 Despliegue de la aplicación móvil

La aplicación móvil se ha desarrollado en Android Studio usando Java.

Requisitos previos

- Android Studio instalado
- Emulador de Android o dispositivo físico
- API REST corriendo y accesible desde la red

Pasos de despliegue

1. Abrir el proyecto en Android Studio.
2. Configurar la URL base de la API en la clase `Api.java` si es necesario.
3. Conectar un dispositivo Android o configurar un emulador.
4. Ejecutar la aplicación desde Android Studio (`Run > Run 'app'`).

La app se conectará a la API para autenticar usuarios, mostrar el feed y gestionar interacciones sociales.

En caso de utilizar un emulador de Android, la dirección localhost debe sustituirse por 10.0.2.2 para acceder a la API que se ejecuta en el equipo anfitrión.

Es necesario que el dispositivo o emulador tenga acceso a la red donde se encuentra la API.

8.4 Despliegue de la aplicación de escritorio

La aplicación de escritorio se ejecuta como una aplicación nativa de Windows y requiere acceso a la API REST para su funcionamiento.

Requisitos previos

- Visual Studio 2022 o superior
- .NET Framework 4.8 o superior
- API REST corriendo y accesible

Pasos de despliegue

1. Abrir la solución en Visual Studio.
2. Configurar la URL base de la API en la clase `ApiService.cs` para asegurar la correcta comunicación con el backend.
3. Compilar el proyecto (`Build > Build Solution`).
4. Ejecutar la aplicación (`Debug > Start Debugging o Ctrl + F5`).

La aplicación de escritorio funcionará como cliente de la API, mostrando el feed y permitiendo crear publicaciones, seguir usuarios y recibir notificaciones.

8.5 Entornos de ejecución

El sistema puede ejecutarse en diferentes entornos:

- **Entorno de desarrollo:** utilizado durante la implementación del proyecto, con todos los servicios ejecutándose en local (API, base de datos y aplicaciones cliente).

- **Entorno de pruebas:** permite validar el correcto funcionamiento antes del despliegue final.

- **Entorno de producción:** destinado a usuarios finales, donde la API se despliega en un servidor accesible públicamente y las aplicaciones cliente se configuran para consumir dicha API.

La separación de entornos permite realizar pruebas sin afectar al sistema en producción y facilita el mantenimiento y evolución del proyecto.

9. Guía de uso y mantenimiento

Esta sección proporciona instrucciones detalladas para ejecutar el proyecto en entornos locales, así como recomendaciones para ampliar funcionalidades y resolver problemas frecuentes. Está pensada para desarrolladores y usuarios avanzados que necesiten mantener o ampliar el sistema.

9.1 Ejecución en local

Para levantar el sistema completo en un entorno local, es necesario ejecutar los cuatro componentes principales: API, aplicación web, aplicación móvil y aplicación de escritorio.

API REST (Java)

Requisitos previos

- Java JDK 17 o superior
- Maven
- MariaDB/MySQL en funcionamiento con la base de datos creada
- Apache Tomcat 9+

Pasos

1. Clonar el repositorio de la API.
2. Construir el proyecto con Maven:

   ```
   mvn clean package
   ```

3. Copiar el archivo `.war` generado en `target/` a la carpeta `webapps/` de Tomcat.
4. Iniciar Tomcat:

   ```
   catalina.bat start       # Windows
   ./catalina.sh start      # Linux/macOS
   ```

5. Verificar la API con un endpoint de prueba:

   ```
   GET http://localhost:8080/Yapi/rest/users/ping
   ```

   Debe devolver pong si la API funciona correctamente.

Aplicación Web

Requisitos previos

- PHP 8+
- Composer
- Servidor web (Apache/Nginx)
- Base de datos MariaDB/MySQL con migraciones aplicadas

Pasos

1. Clonar el proyecto web.
2. Instalar dependencias:

   ```
   composer install
   ```

3. Configurar `.env` con los datos de la base de datos y la URL de la API:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=y_db
DB_USERNAME=root
DB_PASSWORD=
API_URL=http://localhost:8080/Yapi/rest
```

4. Ejecutar migraciones:

```
php artisan migrate
```

5. Iniciar el servidor de desarrollo:

```
php artisan serve
```

6. Acceder desde un navegador:

```
http://localhost:8000
```

Aplicación Móvil

Requisitos previos

- Android Studio
- Emulador o dispositivo físico
- API REST accesible desde la red

Pasos

1. Abrir el proyecto en Android Studio.
2. Configurar la URL base de la API en `Api.java` si es necesario.

   Para emuladores, usar `10.0.2.2` en lugar de `localhost`.

3. Conectar un dispositivo o configurar un emulador.
4. Ejecutar la aplicación (`Run > Run 'app'`).

Aplicación de Escritorio

Requisitos previos

- Visual Studio 2022+
- .NET Framework 4.8+
- API REST accesible

Pasos

1. Abrir la solución en Visual Studio.
2. Configurar la URL de la API en la clase `ApiService.cs`.
3. Compilar el proyecto (`Build > Build Solution`).
4. Ejecutar la aplicación (`Debug > Start Debugging o Ctrl + F5`).

9.2 Añadir funcionalidades

Para ampliar la aplicación, se recomienda seguir estos pasos:

1. **API**
    - Crear nuevos endpoints siguiendo la estructura de controladores existente.
    - Probar los endpoints mediante herramientas como Postman o Rester.
    - Asegurarse de que las nuevas operaciones cumplen con las reglas de seguridad (bcrypt, consultas preparadas, validación de datos).
2. **Web**
    - Crear controladores, vistas y rutas siguiendo el patrón MVC.
    - Conectar las nuevas funcionalidades con los endpoints correspondientes de la API.
3. **Móvil**
    - Añadir nuevas actividades y modelos según la funcionalidad deseada.
    - Actualizar la clase `Api.java` para consumir nuevos endpoints.

4. **Escritorio**
    - Crear nuevos formularios y modelos.
    - Actualizar `ApiService.cs` para consumir la API adecuadamente.

Mantener consistencia entre los distintos clientes es clave para que todas las funcionalidades nuevas estén disponibles en todas las plataformas.

9.3 Problemas frecuentes

Problema	Solución
API no responde	Verificar que Tomcat está corriendo y que el puerto 8080 está libre
Error de conexión a la base de datos	Revisar configuración en .env (web) o parámetros JDBC (API)
Usuario ya registrado	Asegurarse de que el email y username sean únicos antes de registrar
Aplicación móvil no carga datos	Verificar que la URL de la API es accesible desde el dispositivo/emulador
Error al compilar escritorio	Comprobar que el proyecto está en la versión correcta de .NET y que la API es accesible

10. Mejoras futuras

Aunque la aplicación ya cumple con los objetivos principales del proyecto, existen varias oportunidades de mejora y nuevas funcionalidades que podrían implementarse en versiones futuras para aumentar la usabilidad, el rendimiento y la experiencia del usuario.

10.1 Mejoras previstas

1. **Optimización del rendimiento**
    - Cacheo de resultados frecuentes para reducir llamadas repetidas a la API.
2. **Notificaciones reales**
    - Uso de WebSockets o Firebase Cloud Messaging para que las notificaciones lleguen instantáneamente a todas las plataformas.
3. **Endpoints**
    - Adaptación a estándares REST más estrictos

10.2 Nuevas funcionalidades

1. **Reacciones y respuestas a publicaciones**
    - Implementar “likes” o reacciones tipo emoticonos para aumentar la interacción social.
    - Implementar comentarios en publicaciones que a su vez se puedan responder también.
2. **Mensajes privados entre usuarios**
    - Sistema de mensajería interna para conversaciones directas.
3. **Moderación y reportes**
    - Herramientas para reportar contenido inapropiado o spam.

4. **Lista de seguidores y seguidos**
    - Desde un perfil implementar la opción de pulsar en seguidores o seguidos y ver una lista de ellos.
5. **Sistema de búsqueda**
    - Implementar búsqueda de usuarios o publicaciones.
6. **Perfiles privados**
    - Implementar una opción de privacidad del perfil.
    - Implementar un sistema de solicitud de seguimiento para cuentas privadas que permita visualizar sus publicaciones
7. **Multi-idioma**
    - Soporte para idiomas más allá de español e inglés.

10.3 Optimizaciones posibles

- **Seguridad avanzada**
    - Implementar tokens JWT para autenticación en la API.
    - Añadir límites de peticiones (rate limiting) para prevenir abusos.
- **Experiencia de usuario (UX)**
    - Mejorar la interfaz gráfica adaptándola a estándares de diseño modernos.
    - Optimizar la navegación.
- **Escalabilidad**
    - Preparar la API y la base de datos para soportar mayor número de usuarios y publicaciones simultáneas.

Estas mejoras y nuevas funcionalidades representan un plan de evolución del sistema, que puede implementarse de manera incremental sin afectar la estabilidad de la aplicación actual.